

Moduł 4, zadanie 1 – Irysy

```
import pandas as pd
import matplotlib.pyplot as plt

# Konfiguracja: wyświetlaj wszystko
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

df = pd.read_csv('25__iris.csv')
print(df)
print()
```

```
C:\Users\pos\AppData\Local\Programs\Python\Python313\python.exe C:\Users\pos\Documents\Python\Python-lessons\akademia\zadania\mod_4\mod_4_zad_1.py
```

	długość kielicha (sepal length)	szerokość kielicha (sepal width) \
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6
5	5.4	3.9
6	4.6	3.4
7	5.0	3.4
8	4.4	2.9
9	4.9	3.1
10	5.4	3.7
11	4.8	3.4
12	4.8	3.0
13	4.3	3.0
14	5.8	4.0

	długość płatka (petal length)	szerokość płatka (petal width)	\
0	1.4	0.2	
1	1.4	0.2	
2	1.3	0.2	
3	1.5	0.2	
4	1.4	0.2	
5	1.7	0.4	
6	1.4	0.3	
7	1.5	0.2	
8	1.4	0.2	
9	1.5	0.1	
10	1.5	0.2	
11	1.6	0.2	
12	1.4	0.1	
13	1.1	0.1	
14	1.2	0.2	
15	1.5	0.4	

149	5.1	1.8
	Klasa (class)	
0	Iris-setosa	
1	Iris-setosa	
2	Iris-setosa	
3	Iris-setosa	
4	Iris-setosa	
5	Iris-setosa	
6	Iris-setosa	
7	Iris-setosa	
8	Iris-setosa	
9	Iris-setosa	
10	Iris-setosa	
11	Iris-setosa	
12	Iris-setosa	
13	Iris-setosa	

2. Wylistowanie kolumn zastanych i zastąpienie ich własnymi:

```
print('Kolumny:')
print(df.columns)
df.columns = ['Dł_kiel', 'Szer_kiel', 'Dł_płat', 'Szer_płat', 'Klasa']
print()
print('Po zmianie nazw kolumn:')
print(df)
print()
```

```
Kolumny:
Index(['długość kielicha (sepal length)', 'szerokość kielicha (sepal width)',
      'długość płatka (petal length)', 'szerokość płatka (petal width)',
      'klasa (class)'],
      dtype='object')
```

Po zmianie nazw kolumn:

	Dł_kiel	Szer_kiel	Dł_płat	Szer_płat	Klasa
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa

3. Zliczenie wartości dla każdej klasy i zliczenie unikatów w każdej z kolumn:

```
print('Łącznie w każdej klasie jest:')
print(df.groupby('Klasa').size())
print()

print('Unikaty:')
print(df.nunique())
print()
```

```
Łącznie w każdej klasie jest:
Klasa
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
dtype: int64

Unikaty:
Dł_kiel      35
Szer_kiel    23
Dł_płat      43
Szer_płat    22
Klasa         3
dtype: int64
```

4. Praca na danych dot. kolumny długości kielichów:

```
# długości kielichów
lista_dlugosci_kielichow = df["Dł_kiel"].to_list()
print('Lista wartości długości kielichów:', lista_dlugosci_kielichow)

dlugosc_kielicha_max = max(lista_dlugosci_kielichow)
print('Najdłuższy kielich:', dlugosc_kielicha_max)

dlugosc_kielicha_min = min(lista_dlugosci_kielichow)
print('Najkrótszy kielich:', dlugosc_kielicha_min)

potencjal_wzrostu_wzdłuż = dlugosc_kielicha_max - dlugosc_kielicha_min
procentowy_potencjal_wzrostu_wzdłuż = potencjal_wzrostu_wzdłuż / dlugosc_kielicha_min * 100
print('Irys wzdłuż może wzrosnąć o', round(potencjal_wzrostu_wzdłuż, 1), 'cm, czyli o', round(procentowy_potencjal_wzrostu_wzdłuż, 1), 'procent.')
print()
```

Lista wartości długości kielichów: [5.1, 4.9, 4.7, 4.6, 5.0, 5.4, 4.6, 5.0, 4.4, 4.9, 5.4, 4.8, 4.8, 4.3, 5.8, 5.7, 5.4, 5.1, 5.7, 5.1, 5.4, 5.1, 4.6, 5.1, 4.8, 5.0, 5.0, 5.2, 5.2, 4.7, 4.8, 5.4, 5.1]

Najdłuższy kielich: 5.8

Najkrótszy kielich: 4.3

Irys wzdłuż może wzrosnąć o 1.5 cm, czyli o 33.3 procent.

5. Praca na danych dot. kolumny szerokości kielichów:

```
# szerokości kielichów
lista_szerokosci_kielichow = df["Szer-kiel"].to_list()
print('Lista wartości szerokości kielichów:', lista_szerokosci_kielichow)

szerokosci_kielicha_max = max(lista_szerokosci_kielichow)
print('Najszerszy kielich:', szerokosci_kielicha_max)

szerokosci_kielicha_min = min(lista_szerokosci_kielichow)
print('Najwęższy kielich:', szerokosci_kielicha_min)

potencjal_wzrostu_wszerz = szerokosci_kielicha_max - szerokosci_kielicha_min
procentowy_potencjal_wzrostu_wszerz = potencjal_wzrostu_wszerz / szerokosci_kielicha_min * 100
print('Iryś wszerz może wzrosnąć o', round(potencjal_wzrostu_wszerz, 1), 'cm, czyli o', round(procentowy_potencjal_wzrostu_wszerz, 1), 'procent.')
print()
```

```
Lista wartości szerokości kielichów: [3.5, 3.0, 3.2, 3.1, 3.6, 3.9, 3.4, 3.4, 2.9, 3.1, 3.7, 3.4, 3.0, 3.0, 4.0, 4.4, 3.9, 3.5, 3.8, 3.8, 3.4, 3.7, 3.0, 3.3, 3.4, 3.0, 3.4, 3.5, 3.4, 3.2, 3.1, 3.4, 3.1]
Najszerszy kielich: 4.4
Najwęższy kielich: 2.0
Iryś wszerz może wzrosnąć o 2.4 cm, czyli o 120.0 procent.
```

6. Praca na danych dot. kolumny długości płatków – na Data Frame a nie na liście tym razem:

```
# długości płatków - liczone na data frame
najdluzszy_platek = df['Dł_płat'].max()
print('Najdłuższy płatek ma', najdluzszy_platek, 'cm.')

najkrotszy_platek = df['Dł_płat'].min()
print('Najkrótszy płatek ma', najkrotszy_platek, 'cm.')

potencjal_wzrostu_platka_wzdłuż = najdluzszy_platek - najkrotszy_platek
procentowy_potencjal_wzrostu_platka_wzdłuż = potencjal_wzrostu_platka_wzdłuż / najkrotszy_platek * 100
print('Płatek irysa wzdłuż może przyrosnąć o', round(potencjal_wzrostu_platka_wzdłuż, 1), 'cm, czyli o', round(procentowy_potencjal_wzrostu_platka_wzdłuż, 1), 'procent.')
print()
```

Najdłuższy płatek ma 6.9 cm.

Najkrótszy płatek ma 1.0 cm.

Płatek irysa wzdłuż może przyrosnąć o 5.9 cm, czyli o 590.0 procent.

7. Praca na danych dot. kolumny szerokości płatków – na Data Frame a nie na liście tym razem:

```
# szerokości płatków - liczone na data frame
najszerzy_platek = df['Szer_płat'].max()
print('Najszerzy płatek ma', najszerszy_platek, 'cm.')

najwęższy_platek = df['Szer_płat'].min()
print('Najwęższy płatek ma', najwęższy_platek, 'cm.')
potencjal_wzrostu_platka_wszere = najszerszy_platek - najwęższy_platek
procentowy_potencjal_wzrostu_platka_wszere = potencjal_wzrostu_platka_wszere / najwęższy_platek * 100
print('Płatek irysa wszere może przyrosnąć o', round(potencjal_wzrostu_platka_wszere, 1), 'cm, czyli o', round(procentowy_potencjal_wzrostu_platka_wszere, 1), 'procent')
print()
```

Najszerzy płatek ma 2.5 cm.

Najwęższy płatek ma 0.1 cm.

Płatek irysa wszere może przyrosnąć o 2.4 cm, czyli o 2400.0 procent

8. Analiza grup w ramach agregacji:

```
# analiza grup na data frame
print('Zbiorczo - wskaźniki agregujące jako data frame:')
print(
    df.groupby("Klasa")[["Dł_kiel", "Szer_kiel", "Dł_płat", "Szer_płat"]].agg(['min', 'max', 'mean'])
)
```

Zbiorczo - wskaźniki agregujące jako data frame:

	Dł_kiel			Szer_kiel			Dł_płat			
	min	max	mean	min	max	mean	min	max	mean	
Klasa										
Iris-setosa	4.3	5.8	5.006	2.3	4.4	3.418	1.0	1.9	1.464	
Iris-versicolor	4.9	7.0	5.936	2.0	3.4	2.770	3.0	5.1	4.260	
Iris-virginica	4.9	7.9	6.588	2.2	3.8	2.974	4.5	6.9	5.552	

	Szer_płat		
	min	max	mean
Klasa			
Iris-setosa	0.1	0.6	0.244
Iris-versicolor	1.0	1.8	1.326
Iris-virginica	1.4	2.5	2.026

9. Napisalem funkcję, która jest następnie wywoływana w pętli i pokazuje wyliczenia – w tym przypadku średniej dla każdego z wymiarów irysa – dł. i szer. kielicha i odpowiednio płata.

```
# wyliczenie sredniej generyczna funkcja
def wylicz_srednia_wymiaru_dla_klasy(wymiar): # przy wywołaniu funkcji jako 'wymiar' podany będzie string wynikowy ze znanej już listy/obiektu kolumn z wywołania df.columns i usage new *
    # ta zmienna dla podanego wymiaru irysa - dł. lub szer. łodygi, dł. lub szer. kielicha - przechowuje średnią dla zadeklarowanego przy wywołaniu funkcji wymiaru kwiatka
    srednia_wymiaru = df.groupby('Klasa')[wymiar].mean()
    print(srednia_wymiaru) # tu wyświetlamy średnią

# poniższa pętla wywołuje funkcję "wylicz_srednia_wymiaru_dla_klasy", ale najpierw zaglądamy do struktury "df.columns" i leci po jej pozycjach, które podaje funkcji jako parametry.
# Poniżej ostatnią daną "Klasa" bo to jest byt innego typu, stąd -1
for i in range(len(df.columns) - 1):
    wymiar = df.columns[i]
    print('Wyliczam średnią dla:', wymiar)
    wylicz_srednia_wymiaru_dla_klasy(wymiar)
    print()
```

Wyliczam średnią dla: Dł_kiel

Klasa

Iris-setosa 5.006

Iris-versicolor 5.936

Iris-virginica 6.588

Name: Dł_kiel, dtype: float64

Wyliczam średnią dla: Szer_kiel

Klasa

Iris-setosa 3.418

Iris-versicolor 2.770

Iris-virginica 2.974

Name: Szer_kiel, dtype: float64

Wyliczam średnią dla: Dł_płat

Klasa

Iris-setosa 1.464

Iris-versicolor 4.260

Iris-virginica 5.552

Name: Dł_płat, dtype: float64

Wyliczam średnią dla: Szer_płat

Klasa

Iris-setosa 0.244

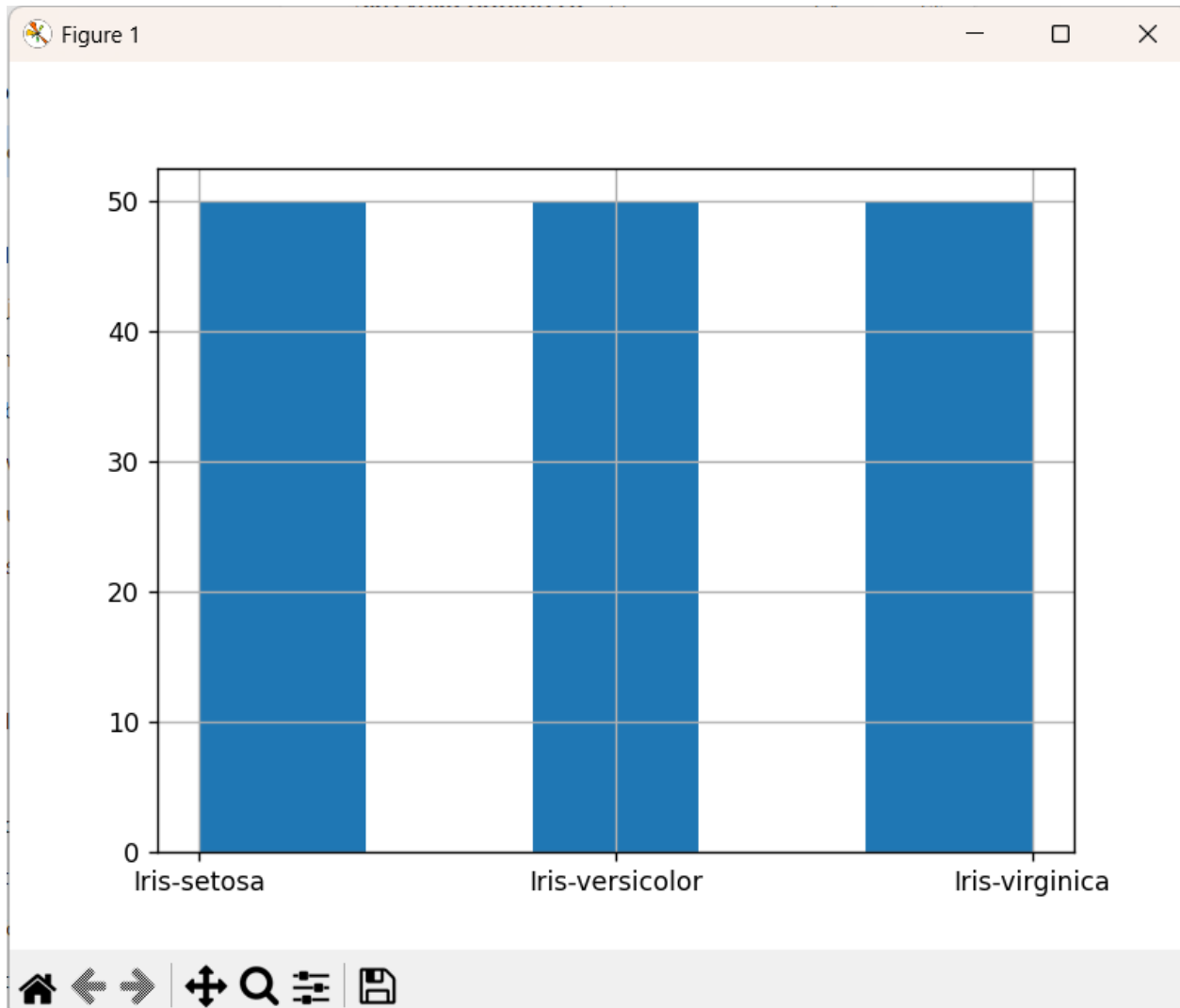
Iris-versicolor 1.326

Iris-virginica 2.026

Name: Szer_płat, dtype: float64

10. Wykresy

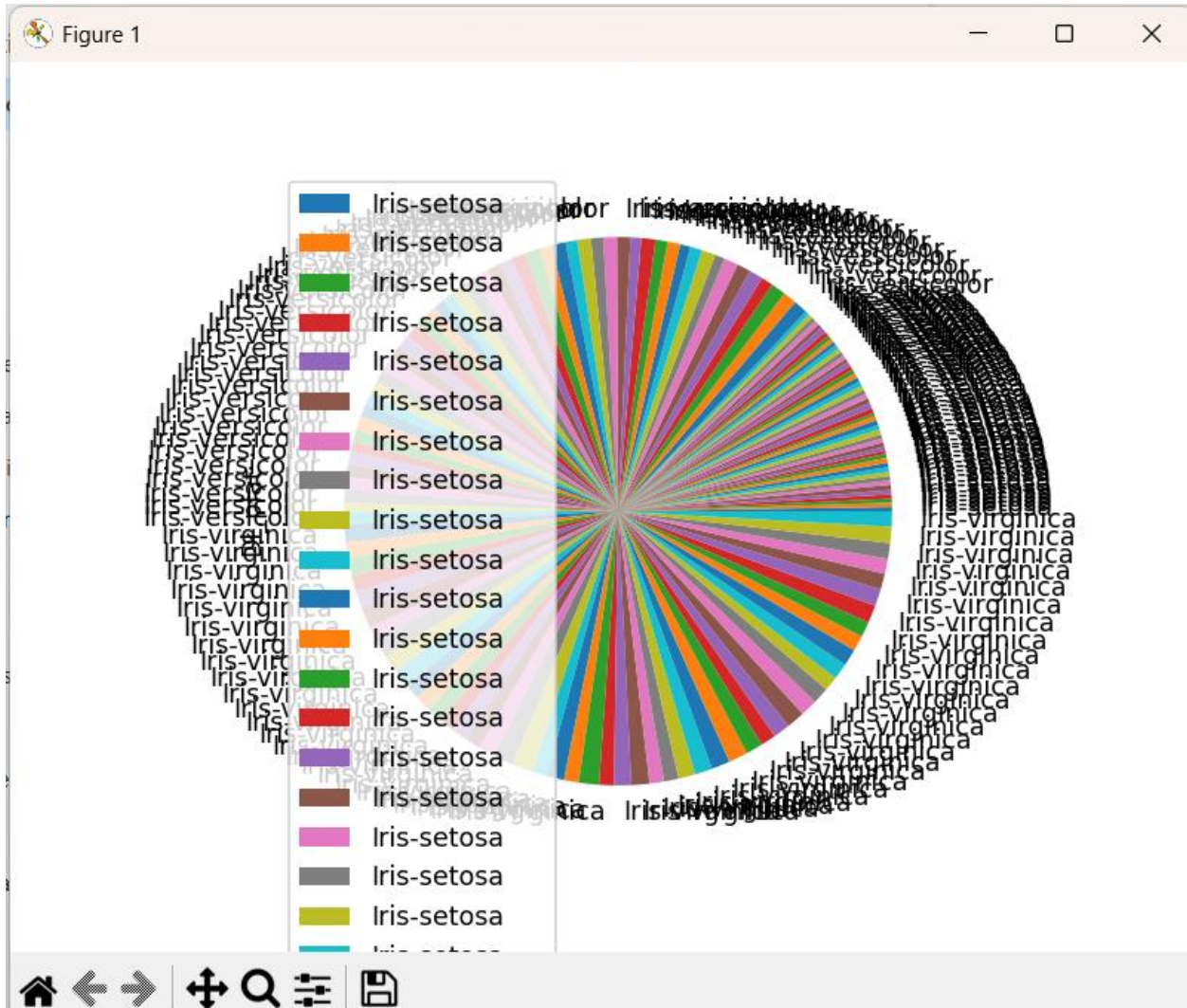
```
# histogram  
df["Klasa"].hist(bins=5)  
plt.show()
```



Podział na pięć binsów przejrzyście pokazuje udział próbek każdej klasy – akurat tutaj jest ich po 50 co sugeruje, że wcześniejsze i dalsze analizy są miarodajne, gdyż nie mamy klasy, której ilość próbek jest nadzwyczajnie duża lub nadzwyczajnie niewielka.

Wykres kołowy:

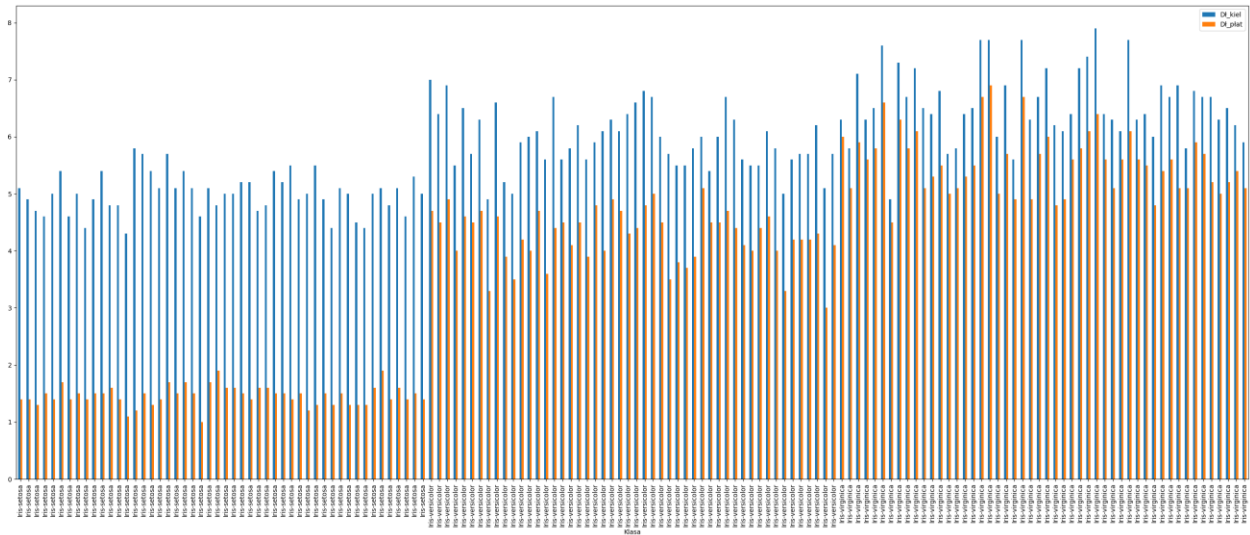
```
# wykres kołowy
df.plot(kind="pie", y="Dł_płat", labels=df['Klasa'], legend=True)
plt.show()
```



Przy zastanej konfiguracji Data Frame program pokazał co umie, czyli po prostu poszalał, ale to sugeruje poprawne wczytanie rzędów i serii danych. Dalszym krokiem byłoby wyłuskanie od biznesu co byłoby dla nich wartościowe i głębsze poznanie modułu wykresów przez developera 😊

Wykres słupkowy:

```
# wykres słupkowy
df.plot(kind="bar", x="Klasa", y=["Dł_kiel", "Dł_płat"])
plt.show()
```



Wykres – pomimo swojej pozornej nieczytelności – jest bardzo czytelny w pokazaniu tendencji wzrostowych dla poszczególnych odmian/typów irysów.

Co można z tego wyczytać? Na przykład przekrojowy zakres rozmiarów kwiatów, co przy prowadzeniu biznesu może wpłynąć na strategię kształtowania cen, gdyż mniejsze egzemplarze wystarczy wpakować do mniejszego opakowania, mniej też miejsca mogą one zająć na ekspozycji, a co za tym idzie: 1) będziemy mogli/musieli zamówić różnego rodzaju/wymiaru opakowania, 2) będziemy różnicować cenę wliczoną za transport w zależności od tej wartości. Otóż większych kwiatów zmieści się mniej na ciężarówce transportowej, a więc trzeba rozważyć konwersję konkretnego wolumenu zamówienia na zysk; do rozważenia jest także większa – a co za tym idzie droższa – skrzynka w automacie paczkowym, gdyż gdzieś na którejś wartości osi Y możemy przekraczać progową wartość dla rozmiaru przesyłki M i zamawiać tylko skrytki L lub w ogóle być zmuszeni do omijania automatów i bardzo duże egzemplarze wysyłać wyłącznie kurierem z dostawą osobistą.

Wykresy punktowe:

```
# wykres punktowy
df.plot(kind="scatter", x="Klasa", y="Dł_kiel")
plt.show()

df.plot(kind="scatter", x="Klasa", y="Szer_kiel")
plt.show()

df.plot(kind="scatter", x="Klasa", y="Dł_płat")
plt.show()

df.plot(kind="scatter", x="Klasa", y="Szer_płat")
plt.show()
```

Figure 1

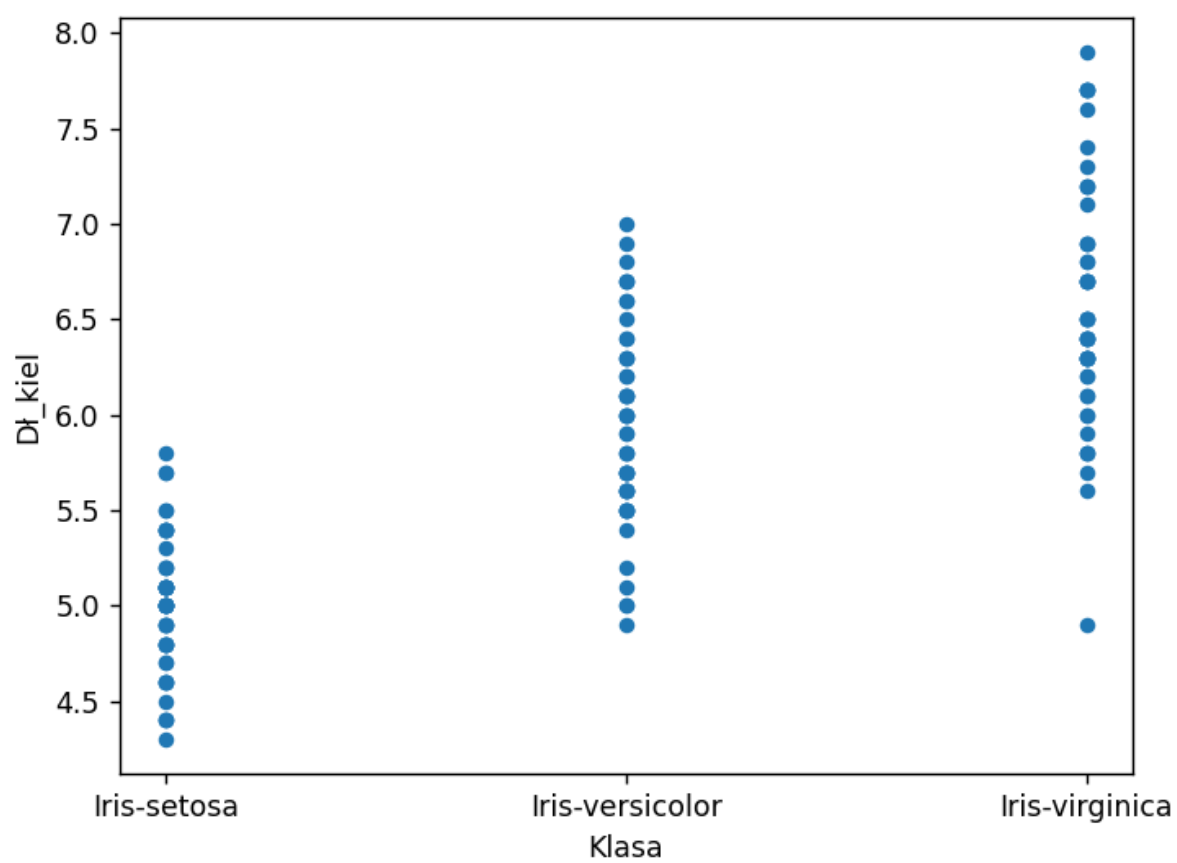




Figure 1

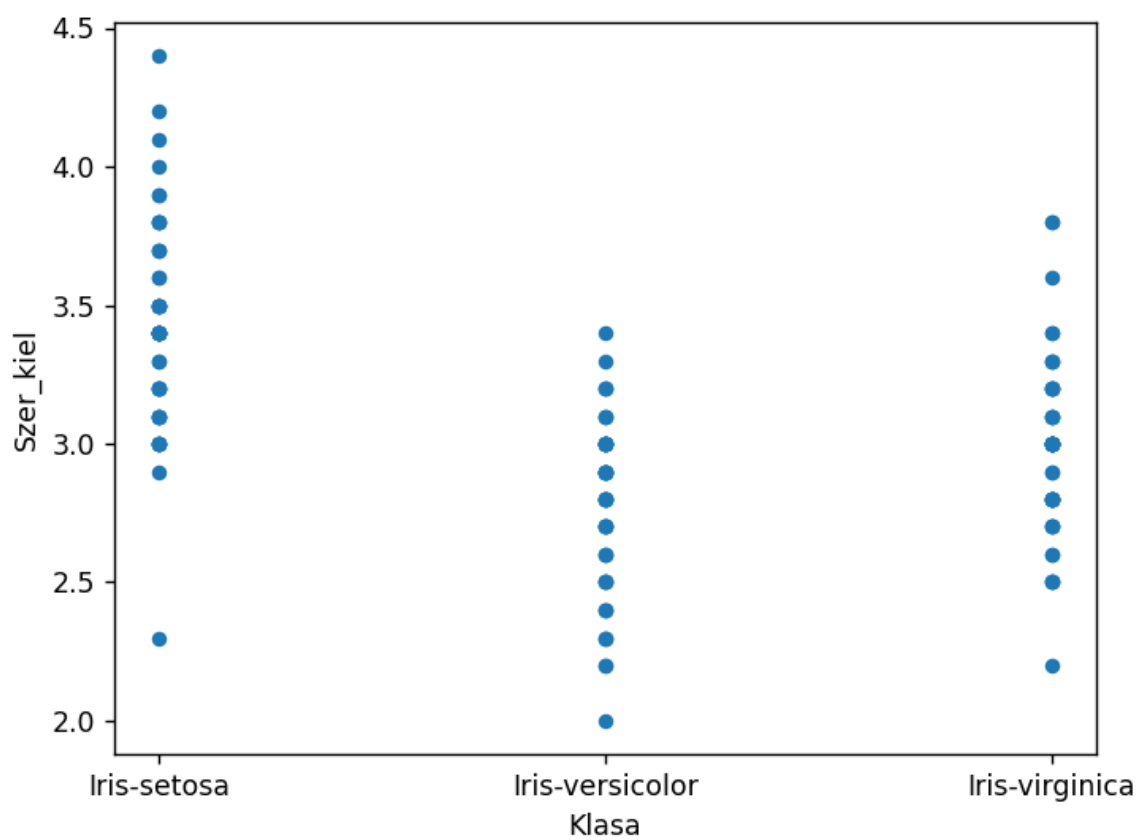


Figure 1

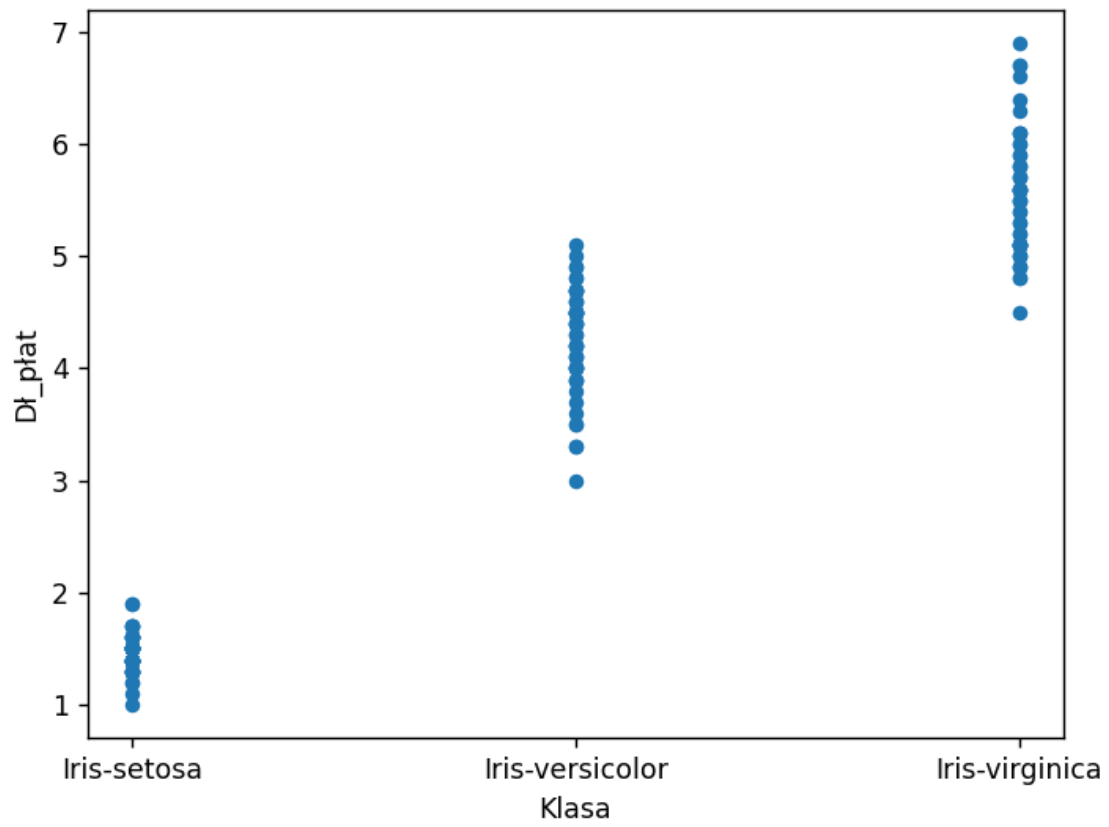
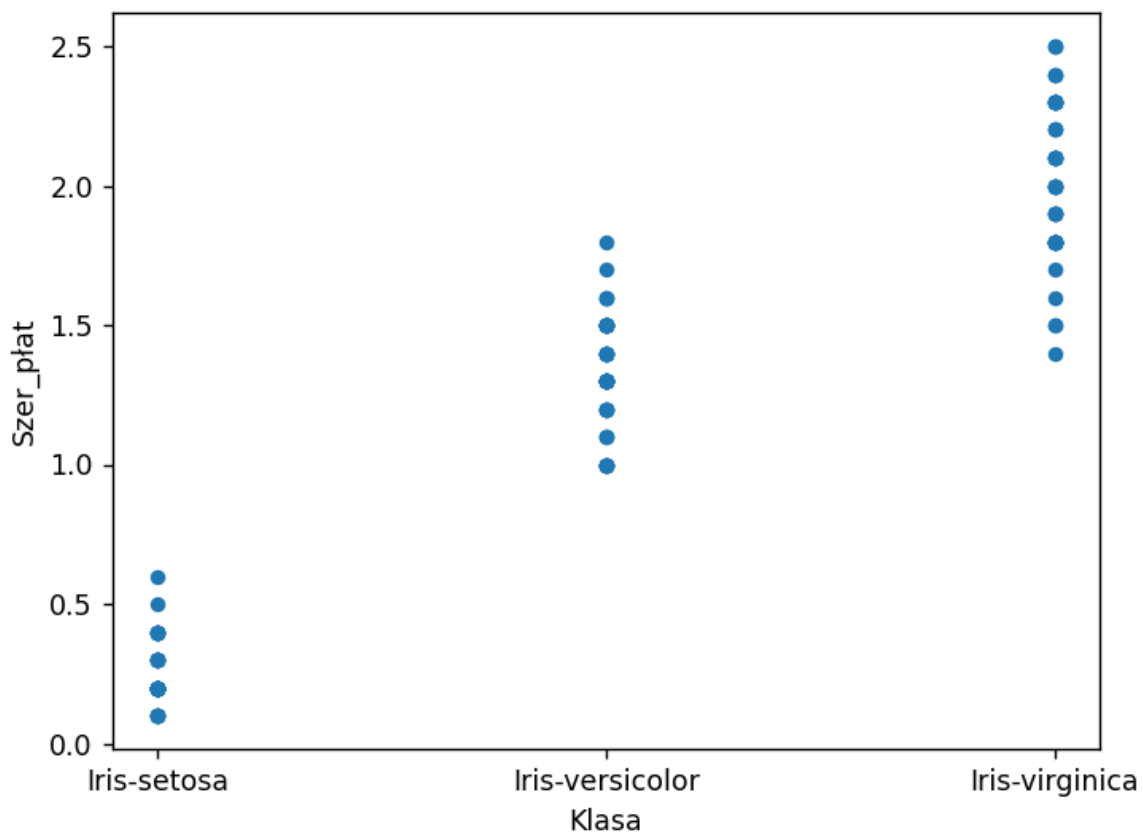


Figure 1

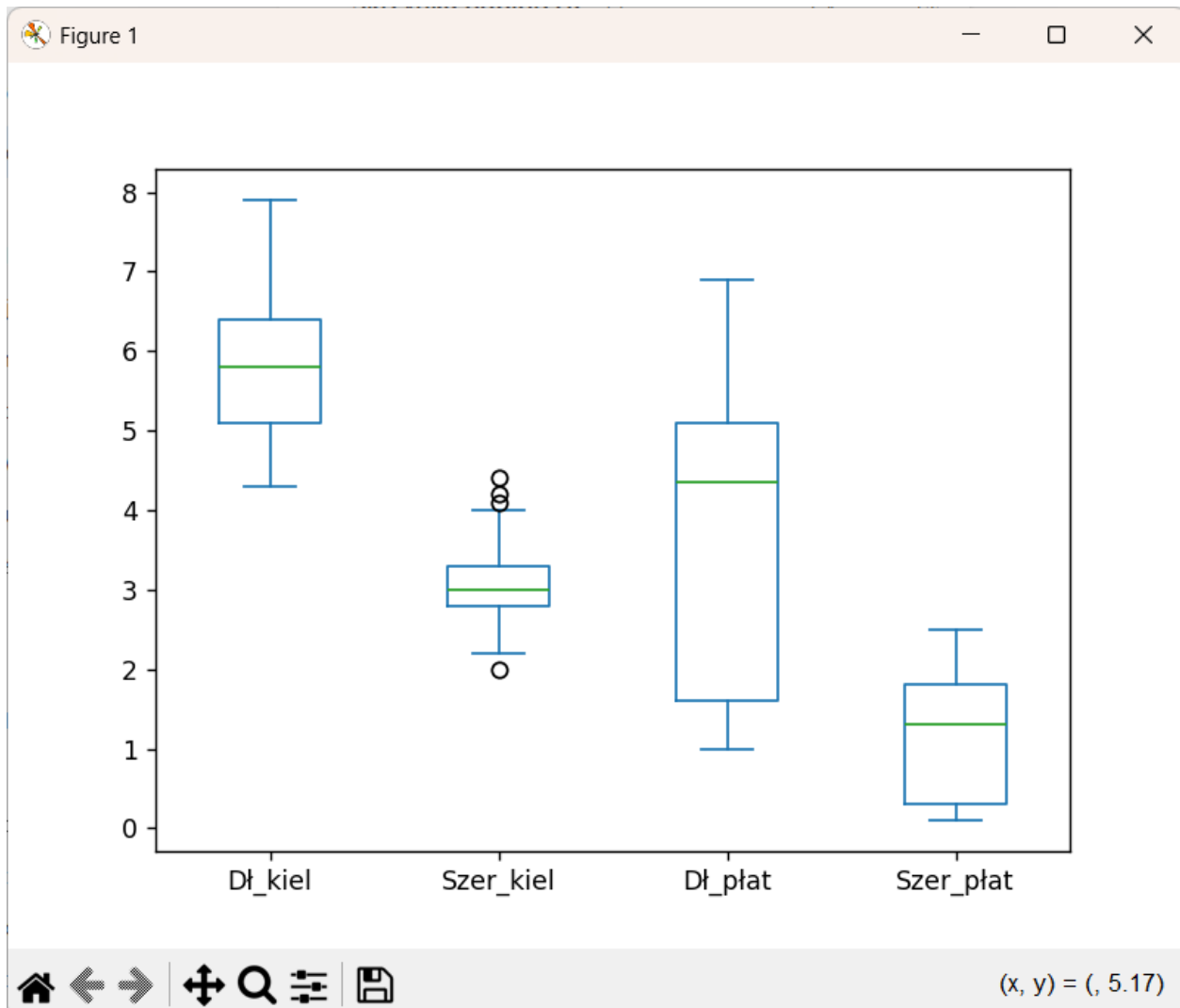


(x, y) = (Iris-virginica, 2.277)

Powyższe wykresy ukazują rozstrzał wartości dla każdego z analizowanych wymiarów każdej z klas irysów. Możemy dokładniej przyjrzeć się wartościom skrajnym i pogłębić analizę w oparciu o poprzedni wykres słupkowy rozważając opakowania 'nie mniejsze niż' i 'nie większe niż'.

Wykres pudełkowy:

```
# wykres pudełkowy
df.plot(kind="box")
plt.show()
```



Ten wykres pokazuje wartości min. i max, oznakowuje kwartyle 1-wszy (percentyl 25-ty), 2-gi (percentyl 50-ty) i 3-ci (percentyl 75-ty), a także wartości odstające dla każdego z analizaowanych wymiarów irysa z poddanej obserwacji zbioru próbek.

Analizę przeprowadzono w IDE PyCharm firmy JetBrains.

Repozytorium dostępne jest pod adresem: https://github.com/racibornio/Python-lessons/blob/master/akademia/zadania/mod_4/mod_4_zad_1.py